

Teoría de la computación — Laboratorio 8

Problema 1

Parte a) Complejidad de tiempo

```
void function(int n) {
    int i, j, k, counter = 0;
    for (i = n/2; i <= n; i++)
        for (j = 1; j+n/2 <= n; j++)
            for (k = 1; k <= n; k = k*2)
                counter++;
}
```

Paso 1 — Ciclo externo (i). Va desde $n/2$ hasta n , es decir, $n - n/2 + 1 \approx n/2$ iteraciones.

Paso 2 — Ciclo intermedio (j). La condición $j + n/2 \leq n$ equivale a $j \leq n/2$, por lo que ejecuta $\approx n/2$ iteraciones. Es independiente de i , así que se multiplica con el ciclo externo.

Paso 3 — Ciclo interno (k). k toma los valores $1, 2, 4, \dots, 2^m$ con $2^m \leq n$, es decir $m \leq \log_2 n$. Ejecuta $\lfloor \log_2 n \rfloor + 1$ iteraciones.

Paso 4 — Total. Los ciclos son independientes, entonces el número de veces que se ejecuta `counter++` es el producto. Para n par, con $m = n/2$:

$$T(n) = (m + 1) \cdot m \cdot (\lfloor \log_2 n \rfloor + 1) = (n/2 + 1)(n/2)(\lfloor \log_2 n \rfloor + 1)$$

Verificación: $n = 10 \rightarrow 6 \cdot 5 \cdot 4 = 120$; $n = 100 \rightarrow 51 \cdot 50 \cdot 7 = 17,850$ (coincide con lo medido).

Paso 5 — Cotas. Para $n \geq 2$: $T(n) \geq (n/2)(n/2)(\log_2 n) = (1/4) n^2 \log n$, y $T(n) \leq n \cdot n \cdot 2 \log n = 2 n^2 \log n$ (pues $n/2 + 1 \leq n$ y $\lfloor \log n \rfloor + 1 \leq 2 \log n$ para $n \geq 2$). Con $c_1 = 1/4$, $c_2 = 2$ y $n_0 = 2$ se cumple $c_1 n^2 \log n \leq T(n) \leq c_2 n^2 \log n$.

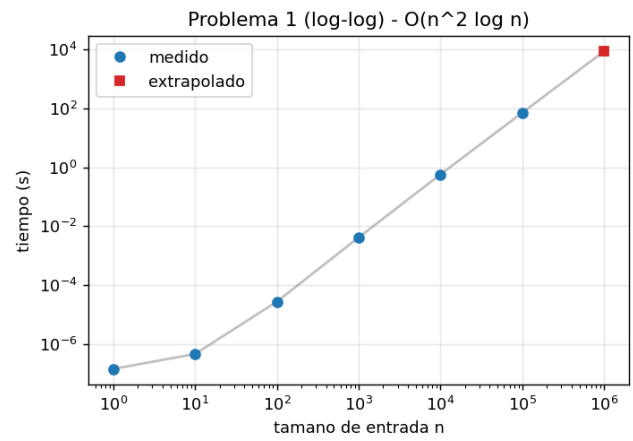
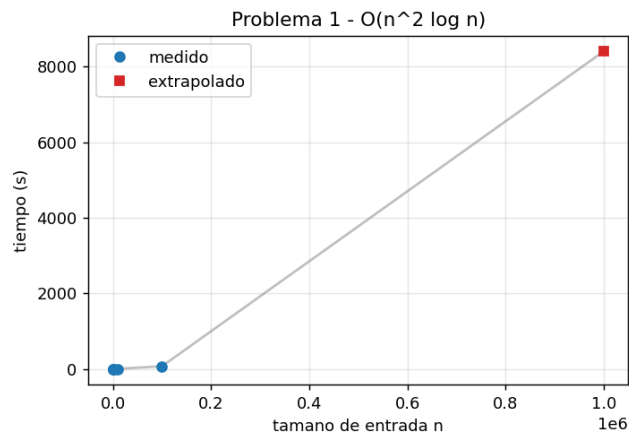
Conclusión: $T(n) = \Theta(n^2 \log n)$, en particular $O(n^2 \log n)$.

Parte b) Profiling

Implementación en C (codigo/problema1.c), compilada con `gcc -O0` y cronometrada con `clock_gettime`.

Para $n \leq 100,000$ el tiempo es medido (mediana de 5 corridas cuando tarda menos de 1 s). Para $n = 1,000,000$ el programa necesitaría $\approx 5 \times 10^{12}$ operaciones (horas de ejecución), así que ese valor se **extrapoló** con $T = c \cdot n^2 \log n$, donde c se ajusta con las mediciones reales.

n	Operaciones	Tiempo (s)	Origen
1	2	0.000000	medido
10	120	0.000000	medido
100	17,850	0.000028	medido
1,000	2,505,000	0.004165	medido
10,000	350,070,000	0.569120	medido
100,000	42,500,850,000	70.020073	medido
1,000,000	—	8402.408744	extrapolado



Interpretación: al multiplicar n por 10 el tiempo crece ≈ 100 -120 veces, que es lo esperado de $n^2 \log n$.

Problema 2

Parte a) Complejidad de tiempo

```
void function(int n) {  
    if (n <= 1) return;  
    int i, j;  
    for (i = 1; i <= n; i++) {  
        for (j = 1; j <= n; j++) {  
            printf("Sequence\n");  
            break;  
        }  
    }  
}
```

Paso 1. La línea 2 (if $n \leq 1$) cuesta $O(1)$.

Paso 2. El ciclo externo se ejecuta n veces.

Paso 3. El ciclo interno, aunque está definido hasta n , tiene un break incondicional: ejecuta printf una vez y sale. Por lo tanto cada iteración del externo cuesta $O(1)$ (no n).

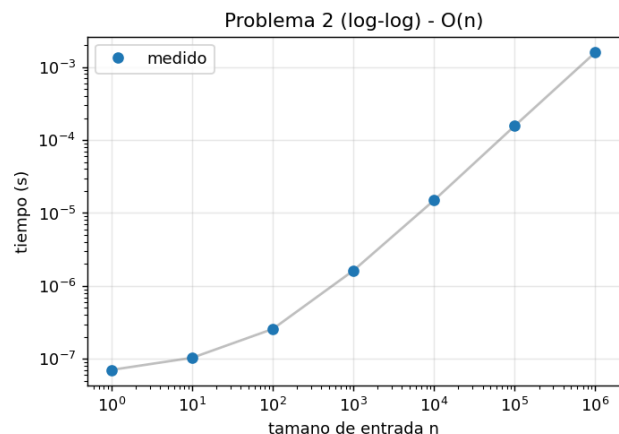
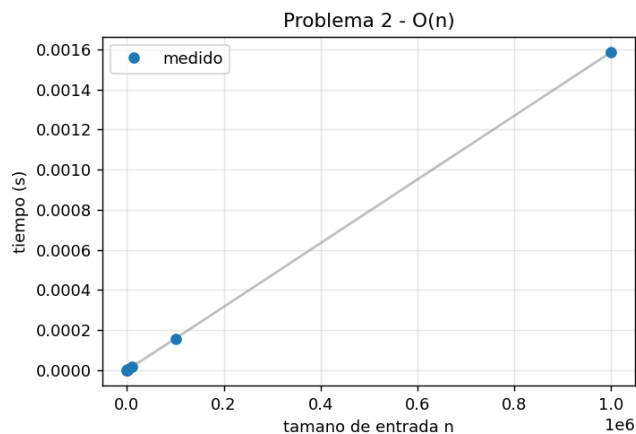
Paso 4. $T(n) = c_0 + n \cdot (c_1 + c_2) = an + b$, con a, b constantes.

Conclusión: $T(n) = \Theta(n)$, es decir $O(n)$. (Un análisis superficial daría $O(n^2)$, pero el break lo reduce a lineal.) El número de líneas impresas es exactamente $n - 1 + 1 = n$ para $n > 1$.

Parte b) Profiling

Programa en C (codigo/problema2.c). Para medir el algoritmo y no la consola, el printf se sustituye por un contador (counter++); el número de “impresiones” es el valor de la columna Operaciones.

n	Operaciones	Tiempo (s)	Origen
1	0	0.000000	medido
10	10	0.000000	medido
100	100	0.000000	medido
1,000	1,000	0.000002	medido
10,000	10,000	0.000015	medido
100,000	100,000	0.000156	medido
1,000,000	1,000,000	0.001586	medido



Interpretación: el tiempo crece ≈ 10 veces cuando n crece 10 veces $\rightarrow$ comportamiento lineal.

Problema 3

Parte a) Complejidad de tiempo

```
void function(int n) {
    int i, j;
    for (i = 1; i <= n/3; i++)
        for (j = 1; j <= n; j += 4)
            printf("Sequence\n");
}
```

Paso 1 — Ciclo externo. i va de 1 a $n/3$ con incremento 1: $\lfloor n/3 \rfloor$ iteraciones.

Paso 2 — Ciclo interno. $j = 1, 5, 9, \dots$, con incremento 4 hasta n : $\lfloor n/4 \rfloor$ iteraciones.

Paso 3 — Total. Los ciclos son independientes:

$$T(n) = \lfloor n/3 \rfloor \cdot \lfloor n/4 \rfloor \approx (n/3)(n/4) = n^2/12$$

Verificación: $n = 100 \rightarrow 33 \cdot 25 = 825$ (coincide con lo medido).

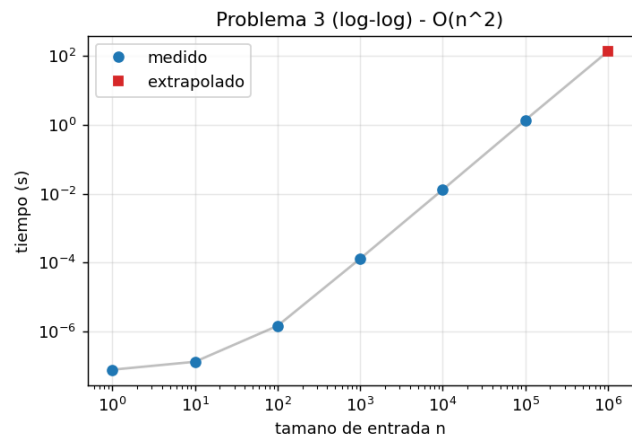
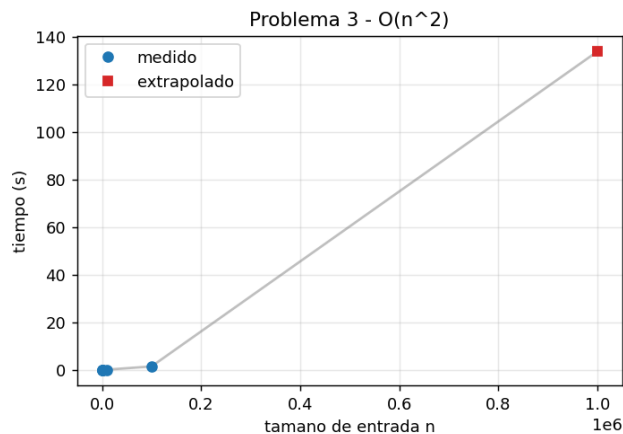
Paso 4 — Cotas. Para $n \geq 12$: $n^2/20 \leq T(n) \leq n^2/12 + n$, y por tanto $c_1 n^2 \leq T(n) \leq c_2 n^2$ con $c_1 = 1/20$, $c_2 = 1$.

Conclusión: $T(n) = \Theta(n^2)$, es decir $O(n^2)$. Las constantes $1/3$ y $1/4$ no cambian el orden.

Parte b) Profiling

Programa en C (codigo/problema3.c); igual que en el Problema 2, el printf se sustituye por un contador. $n = 1,000,000$ implica $\approx 8.3 \times 10^{10}$ iteraciones; ese valor se **extrapoló** con $T = c \cdot n^2$ a partir de las mediciones.

n	Operaciones	Tiempo (s)	Origen
1	0	0.000000	medido
10	9	0.000000	medido
100	825	0.000001	medido
1,000	83,250	0.000130	medido
10,000	8,332,500	0.013089	medido
100,000	833,325,000	1.365353	medido
1,000,000	—	133.710788	extrapolado



Interpretación: al multiplicar n por 10 el tiempo crece ≈ 100 veces $\rightarrow$ comportamiento cuadrático.

Problema 4

Mejor caso, caso promedio y peor caso. Sea n el tamaño del arreglo y se cuenta el número de comparaciones.

4.1 Búsqueda lineal (Linear Search)

```
for i = 0 .. n-1:
    if A[i] == x: return i
return -1
```

Mejor caso: x está en la primera posición $\rightarrow$ 1 comparación $\rightarrow O(1)$.

Peor caso: x está en la última posición o no está $\rightarrow n$ comparaciones $\rightarrow O(n)$.

Caso promedio: si x está y cada posición es equiprobable (probabilidad $1/n$), el número esperado de comparaciones es $(1/n)(1 + 2 + \dots + n) = (1/n) \cdot n(n+1)/2 = (n+1)/2 \rightarrow \Theta(n)$. (Si x puede no estar, el promedio sigue siendo proporcional a n .)

4.2 Búsqueda binaria (Binary Search, arreglo ordenado)

```
lo = 0; hi = n-1
while lo <= hi:
    mid = (lo+hi)/2
    if A[mid] == x: return mid
    elif A[mid] < x: lo = mid+1
    else: hi = mid-1
return -1
```

Mejor caso: x está justo en el elemento central $\rightarrow$ 1 comparación $\rightarrow O(1)$.

Peor caso: cada paso reduce el rango a la mitad: $T(n) = T(n/2) + c$, que resuelve a $T(n) = c(\lfloor \log_2 n \rfloor + 1) \rightarrow O(\log n)$.

Caso promedio: en un árbol de decisión completo hay 1 elemento que se halla en 1 comparación, 2 en 2, 4 en 3, ..., 2^{k-1} en k comparaciones. Con $n \approx 2^h - 1$: promedio = $(1/n) \sum_{k=1..h} k \cdot 2^{k-1} = ((h-1)2^h + 1)/n \approx \log_2(n+1) - 1 \rightarrow \Theta(\log n)$.

4.3 Quick Sort

```
QuickSort(A, lo, hi):
    if lo < hi:
        p = Partition(A, lo, hi) // costo  $\Theta(n)$ 
        QuickSort(A, lo, p-1); QuickSort(A, p+1, hi)
```

Mejor caso: el pivote siempre divide en dos mitades iguales: $T(n) = 2T(n/2) + \Theta(n)$. Por el Teorema Maestro ($a = 2$, $b = 2$, $f(n) = n = n^{\log_2 2}$, caso 2) $\rightarrow \Theta(n \log n)$. Equivalente: $\log n$ niveles de recursión con trabajo n por nivel.

Peor caso: el pivote es siempre el mínimo o el máximo (p. ej., arreglo ya ordenado con pivote en un extremo): $T(n) = T(n-1) + \Theta(n) = \sum_{k=1..n} k = n(n+1)/2 \rightarrow \Theta(n^2)$, es decir $O(n^2)$.

Caso promedio: con todas las posiciones del pivote equiprobables ($1/n$):

$$T(n) = (n-1) + (1/n) \sum_{k=0..n-1} [T(k) + T(n-1-k)] = (n-1) + (2/n) \sum_{k=0..n-1} T(k)$$

Multiplicando por n , restando la ecuación para $n-1$ y simplificando: $nT(n) - (n-1)T(n-1) = 2T(n-1) + 2(n-1)$, es decir $T(n)/(n+1) = T(n-1)/n + 2(n-1)/(n(n+1))$. Al telescoper se obtiene $T(n) \approx 2(n+1)H_n \approx 2n \ln n \approx 1.39 n \log_2 n \rightarrow \Theta(n \log n)$.

Algoritmo	Mejor caso	Caso promedio	Peor caso
Búsqueda lineal	$O(1)$	$\Theta(n)$	$O(n)$
Búsqueda binaria	$O(1)$	$\Theta(\log n)$	$O(\log n)$
Quick Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$

Problema 5

a) Si $f(n) = \Theta(g(n))$ y $g(n) = \Theta(h(n))$, entonces $h(n) = \Theta(f(n))$.

VERDADERO. Por definición, $f = \Theta(g)$ significa que existen $c_1, c_2 > 0$ y n_1 tales que $c_1 g(n) \leq f(n) \leq c_2 g(n)$ para $n \geq n_1$; análogamente $d_1 h(n) \leq g(n) \leq d_2 h(n)$ para $n \geq n_2$.

Combinando para $n \geq \max(n_1, n_2)$: $c_1 d_1 h(n) \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \leq c_2 d_2 h(n)$. Luego $f = \Theta(h)$ (transitividad). Como Θ es simétrica (si $f = \Theta(h)$, entonces $(1/(c_2 d_2)) f \leq h \leq (1/(c_1 d_1)) f$), se concluye $h = \Theta(f)$. Θ es una relación de equivalencia.

b) Si $f(n) = O(g(n))$ y $g(n) = O(h(n))$, entonces $h(n) = \Omega(f(n))$.

VERDADERO. De $f \leq c \cdot g$ para $n \geq n_1$ y $g \leq d \cdot h$ para $n \geq n_2$ se obtiene $f(n) \leq c \cdot d \cdot h(n)$ para $n \geq \max(n_1, n_2)$, es decir $f = O(h)$ (transitividad de O). Por la simetría traspuesta ($f = O(h) \Leftrightarrow h = \Omega(f)$): $h(n) \geq (1/(cd)) f(n)$, así que $h = \Omega(f)$.

c) $f(n) = \Theta(n^2)$ para el tiempo de ejecución del programa Python.

FALSO. El error es suponer que cada $S.add(atupla[i:j])$ cuesta $O(1)$. En realidad, con la rebanada $atupla[i:j]$ se debe: (1) **copiar** $j - i$ elementos para crear la nueva tupla $\rightarrow \Theta(j - i)$, y (2) **calcular su hash**, que recorre todos los elementos de la tupla $\rightarrow \Theta(j - i)$ (el hash de una tupla no se cachea). Insertarla en el set es entonces $\Theta(j - i)$ (las tuplas son todas distintas, así que las comparaciones por igualdad son raras).

El costo total es:

$$f(n) = \sum_{i=0..n-1} \sum_{j=i+1..n-1} (j - i) = \sum_{d=1..n-1} d \cdot (n - d)$$

(hay $n - d$ pares (i, j) con $j - i = d$). Entonces:

$$\sum d(n - d) = n \cdot n(n-1)/2 - (n-1)n(2n-1)/6 = n(n-1)[3n - (2n-1)]/6 = \mathbf{n(n-1)(n+1)/6 = (n^3 - n)/6}$$

Por lo tanto $f(n) = \Theta(n^3)$, y no $\Theta(n^2)$ (los $n(n-1)/2$ pares sí son cuadráticos, pero cada uno cuesta en promedio $\approx n/3$). También el espacio usado es $\Theta(n^3)$, pues se almacenan todas esas tuplas.